# 1. Method: Meaning, Declaration & Invocation

BCA study notes | Simple English • Java examples • Diagrams

## Meaning

A **method** is a block of code that performs a particular task. It helps us reuse code and keep a program organized.

## Declaration

A method declaration contains an access modifier (optional), return type, method name and parameters. The body is written inside curly braces.

## Invocation

Invocation means calling or using a method. A method runs when it is called, directly or through an object.

## Java code example

```
class Demo {
    static void greet() {
        System.out.println("Hello!");
    }
    public static void main(String[] args) {
        greet();                // method invocation
    }
}
```

## Concept diagram

# 2. Parameters & Return Types

BCA study notes | Simple English • Java examples • Diagrams

## Parameters

*Parameters are variables written in the method definition. Arguments are the actual values passed when the method is called.*

## Return type

*The return type tells what kind of value a method sends back. Use **void** when the method does not return a value.*

## Return statement

*A non-void method must return a compatible value on every possible path.*

## Java code example

```
static int add(int a, int b) { // a and b are parameters
    return a + b;
}
public static void main(String[] args) {
    int answer = add(5, 3);      // 5 and 3 are arguments
    System.out.println(answer);  // 8
}
```

# 3. Pass-by-Value in Java

BCA study notes | Simple English • Java examples • Diagrams

## Main rule

*Java is always **pass-by-value**. A method receives a copy of the value supplied by the caller.*

## Primitive values

*For int, double and other primitive types, the copied value is independent. Changing the parameter does not change the caller's variable.*

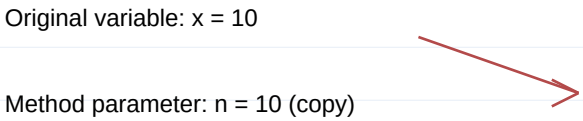
## Object references

*For objects, the copied value is a reference. The method can change the object's contents, but assigning the parameter to a new object does not change the caller's reference.*

## Java code example

```
static void change(int n) {
    n = 99;
}
public static void main(String[] args) {
    int x = 10;
    change(x);
    System.out.println(x); // 10
}
```

## Concept diagram



# 4. Static vs Instance Methods

BCA study notes | Simple English • Java examples • Diagrams

## Static method

*A static method belongs to the class. It can be called using the class name and does not need an object.*

## Instance method

*An instance method belongs to an object. Create an object to call it. It can directly use instance variables and other instance methods.*

## Important rule

*A static method cannot directly access instance members because it has no current object (no this reference).*

## Java code example

```
class Student {
    static void college() {
        System.out.println("SRMU");
    }
    void study() {
        System.out.println("Java");
    }
}
Student.college();       // static call
Student s = new Student();
s.study();               // instance call
```

# 5. Method Overloading & Rules

BCA study notes | Simple English • Java examples • Diagrams

## Meaning

*Method overloading means defining multiple methods with the same name but different parameter lists in the same class.*

## Ways to overload

*Change the number of parameters, their data types, or their order.*

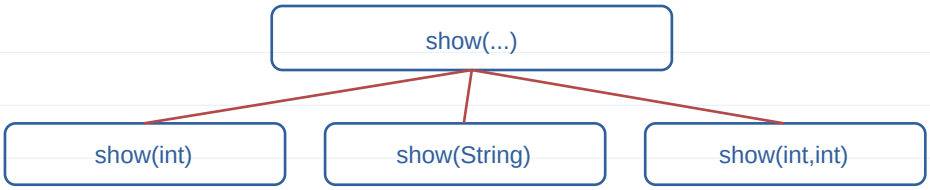
## Rules

*Changing only the return type is not enough. Parameter names do not matter. Overloaded methods may have different return types if their parameter lists are already different.*

## Java code example

```
static int sum(int a, int b) {
    return a + b;
}
static int sum(int a, int b, int c) {
    return a + b + c;
}
static double sum(double a, double b) {
    return a + b;
}
```

## Concept diagram



# 6. Compile-Time Polymorphism

BCA study notes | Simple English • Java examples • Diagrams

## Meaning

*Polymorphism means 'many forms'. In compile-time polymorphism, the compiler selects the method to call using the method name and argument types.*

## Connection

*Method overloading is a common example of compile-time polymorphism.*

## Benefit

*It allows one meaningful method name to perform similar tasks with different kinds or numbers of inputs.*

## Java code example

```
class Printer {
    void print(int n) {
        System.out.println(n);
    }
    void print(String text) {
        System.out.println(text);
    }
}
Printer p = new Printer();
p.print(25);          // selects print(int)
p.print("Java");      // selects print(String)
```

# 7. Varargs & Overloading

BCA study notes | Simple English • Java examples • Diagrams

## Varargs meaning

*Varargs allows a method to accept zero or more arguments of the same type. Syntax: **type... name**.*

## Rules

*Only one varargs parameter is allowed in a method, and it must be the last parameter. Inside the method, it behaves like an array.*

## Overloading

*A fixed-parameter method is generally preferred over a varargs method when both are applicable to the same call.*

## Java code example

```
static int total(int... nums) {
    int sum = 0;
    for (int n : nums) sum += n;
    return sum;
}
total();              // 0
total(2, 3);          // 5
total(1, 2, 3, 4);    // 10
```

# 8. Recursion & Base Condition

BCA study notes | Simple English • Java examples • Diagrams

## Recursion

*Recursion occurs when a method calls itself to solve a smaller version of a problem.*

## Base condition

*The base condition stops recursion. Without it, calls may continue until a StackOverflowError occurs.*

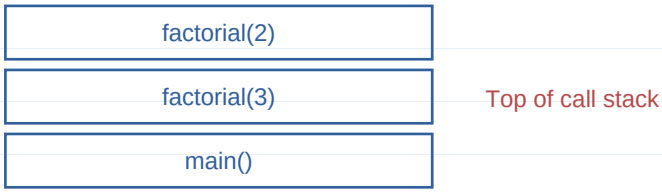
## Recursive step

*Each call should move closer to the base condition, for example by reducing n by 1.*

## Java code example

```
static int factorial(int n) {
    if (n <= 1) return 1;       // base condition
    return n * factorial(n - 1);
}
// factorial(4) = 4 * 3 * 2 * 1 = 24
```

## Concept diagram



# 9. Recursion & Call Stack

BCA study notes | Simple English • Java examples • Diagrams

## Call stack

*The call stack stores information about active method calls, including local variables and where execution should return.*

## Working

*Each recursive call creates a new stack frame. When the base condition is reached, calls return one by one in reverse order.*

## Careful

*Very deep recursion uses more stack memory. For large inputs, an iterative loop may be safer.*

## Java code example

```
factorial(3)
  = 3 * factorial(2)
  = 3 * 2 * factorial(1)
  = 3 * 2 * 1
  = 6
// Calls return in reverse order.
```

# 10. Quick Revision & Complete Example

BCA study notes | Simple English • Java examples • Diagrams

## Remember

*Declaration defines a method; invocation calls it; parameters receive argument values; return sends a result.*

## Exam points

*Java is pass-by-value. Overloading is compile-time polymorphism. Varargs uses ... and must be last. Recursion needs a base condition.*

## Common errors

*Do not overload by return type alone. Do not call an instance method from static context without an object. Avoid recursion without a stopping condition.*

## Java code example

```
class Calculator {
    static int multiply(int a, int b) {
        return a * b;
    }
    static int multiply(int a, int b, int c) {
        return a * b * c;
    }
    static int addAll(int... values) {
        int sum = 0;
        for (int v : values) sum += v;
        return sum;
    }
    public static void main(String[] args) {
        System.out.println(multiply(4, 5));
        System.out.println(addAll(1, 2, 3));
    }
}
```

## Quick revision table

| Topic | Remember |
|---|---|
| Overloading | Same name, different parameter list |
| Varargs | type... name; must be last |
| Recursion | Base condition + smaller call |
| Static method | Called using class name |
| Pass-by-value | Java always passes a copy |